

Principais estruturas de dados: notação e complexidade

Estruturas de dados são formas de organizar os dados em memória. Um **tipo abstrato de dados** é um conjunto de operações que podem ser realizadas nos dados. Um mesmo tipo abstrato de dados pode ter **várias implementações possíveis**, e estas implementações **afetam o tempo de execução** das operações. Portanto, ao utilizar tipos abstratos de dados no algoritmo que estamos projetando, precisamos decidir quais implementações destes tipos abstratos de dados devem ser utilizadas de modo a deixar nosso algoritmo mais eficiente.

Em cada seção a seguir temos um **tipo abstrato**, e indicado entre **parênteses a implementação** deste tipo abstrato que será discutida. Nos pseudocódigos ao longo da disciplina vou assumir que esta implementação indicada entre parênteses é a **implementação adotada para o tipo abstrato**, a menos que outra implementação seja informada na descrição do algoritmo.

Para cada tipo abstrato é fornecida uma tabela contendo o **nome da operação**, um exemplo da **sintaxe** adotada para uso da operação, e a **complexidade da operação** na implementação discutida. Para simplificar a implementação, a sintaxe adotada é idêntica à utilizada na linguagem python. Podemos utilizar outras sintaxes nos pseudocódigos, mas neste caso é necessário deixar claro qual é a operação e o tipo abstrato. Além disso, se não utilizar a convenção indicada aqui, durante a determinação da complexidade é necessário informar qual implementação do tipo abstrato será considerada na análise.

Obs.: Para o **comando de atribuição** podemos usar os símbolos $=$ ou $\leftarrow$, mas se utilizar o símbolo $=$ é melhor fazer o teste de igualdade utilizando $==$. Os **índices de uma lista** podem ser indicados entre colchetes, como em $L[1..n]$, mas se não for informado vamos assumir que elas são indexadas começando de zero.

6.1 Lista (com array dinâmico)

Uma lista é uma **sequência de elementos que podem ser acessados pela sua posição** na sequência. O valor n representa o tamanho (número de elementos) da lista. A menos que seja indicado o contrário, as posições começam em 0 e vão até $n - 1$.

Operação	Exemplo	Complexidade
Cria vazia	$l = []$	$O(1)$
Cria inicializada	$l = [v_1, \dots, v_n]$	$O(n)$
Acessa posição	$v = l[i]$	$O(1)$
Altera posição	$l[i] = v$	$O(1)$
Inserir na posição	$l[i:i] = [v]$	$O(n)$
Inserir no final	$l.append(v)$	$O(1)$
Remove da posição	$v = l.pop(i)$	$O(n)$
Remove do final	$v = l.pop()$	$O(1)$
Remove valor	$l.remove(v)$	$O(n)$
Contido (in, not in)	$v \text{ in } l$	$O(n)$
Posição do valor	$i = l.index(v)$	$O(n)$
Inverte ordem	$l.reverse()$	$O(n)$
Mínimo	$v = \min(l)$	$O(n)$
Máximo	$v = \max(l)$	$O(n)$
Ordena	$l.sort()$	$O(n \log n)$
Compara ($==$, $!=$)	$l1 == l2$	$O(n)$
Concatena	$l = l1 + l2$	$O(n)$
Tamanho	$len(l)$	$O(1)$
Iteração	$for v \text{ in } l$	$O(n)$
Cópia	$l1 = l.copy()$	$O(n)$
Fatia (slice)	$l1 = l[a:b]$	$O(b - a)$
Repete	$l1 = k * l$	$O(kn)$

- “Inserir na posição”: acrescenta elementos com valor v da posição i , mantendo os elementos que já estavam na lista. Note que esta operação acrescenta o elemento no meio da lista, deslocando os elementos originais. Se quiser apenas alterar o valor em uma determinada posição, use a operação “Altera posição”.
- “Remove valor”: remove a primeira ocorrência do valor v na lista, ou lança uma exceção se v não for encontrado. No pseudocódigo basta testar se a operação retorna “Não encontrado”.

- “Posição do valor”: retorna a primeira posição que contém o valor v , ou lança uma exceção se v não for encontrado. No pseudocódigo basta testar se o valor retornado é igual a “Não encontrado”.
- “Repete”: consiste em concatenar k cópias da lista.
- “Fatia” e “Repete” criam cópias da lista.
- As complexidades decorrem do fato das listas em python serem implementadas com arrays dinâmicos.
- “Mínimo” e “Máximo” podem ser realizadas em $O(1)$ se não permitirmos remoções. Para isso, basta atualizar duas variáveis que guardam os valores mínimo e o máximo sempre que uma inserção for feita. Note que a manutenção destas variáveis deixa de ter tempo constante se permitirmos remoções.

Tuplas

Tuplas são **listas imutáveis**, ou seja, cujo tamanho e referências dos elementos não podem ser modificados. A sintaxe das tuplas é similar à sintaxe das listas, exceto que usamos parênteses ao invés de colchetes. As tuplas **suportam todas as operações de lista que não provocam modificação**. As tuplas são muito utilizadas como **retorno de funções** (permite retornar vários valores), como **elementos de estruturas de dados** (vários valores armazenados em um mesmo elemento), e para **inicializar várias variáveis** com um único comando de atribuição.

6.2 Pilha, fila, deque (com lista encadeada)

Uma **pilha** é uma lista em que os elementos são **inseridos e removidos em apenas um dos extremos da lista**. Vimos que podemos inserir e remover no final de uma lista em $O(1)$, o que nos permite concluir que as operações do tipo pilha podem ser implementadas em **tempo constante utilizando as operações de lista** $l.append(v)$ e $v \leftarrow l.pop()$.

Uma **fila** é uma lista em que os elementos são **inseridos em um dos extremos e são removidos no outro extremo**. Como a inserção e a remoção no início da lista são operações com tempo $O(n)$, concluímos que estas operações são **ineficientes se implementadas com listas**.

É possível **inserir e remover no início e no final de uma lista em tempo constante** utilizando uma **lista duplamente encadeada**. O preço que pagamos por utilizar uma lista encadeada é que o acesso a uma posição no meio da lista passa a ter tempo linear (ao invés de constante), mas em uma fila acessamos apenas elementos que estão nos extremos. Na linguagem python, uma lista duplamente encadeada é implementada no módulo `collections.deque`, cujas operações são apresentadas abaixo, com destaque para as complexidades que diferem do tipo lista.

Deque

Importação do módulo: `from collections import deque`

Operação	Exemplo	Complexidade
Cria vazio	<code>q = deque([])</code>	$O(1)$
Cria inicializado	<code>q = deque([v1, ..., vn])</code>	$O(n)$
Acessa posição	<code>v = q[i]</code>	$O(n)$
Altera posição	<code>q[i] = v</code>	$O(n)$
Inserir no início	<code>q.appendleft(v)</code>	$O(1)$
Inserir no final	<code>q.append(v)</code>	$O(1)$
Remove do início	<code>v = q.popleft()</code>	$O(1)$
Remove do final	<code>v = q.pop()</code>	$O(1)$
Remove valor	<code>q.remove(v)</code>	$O(n)$
Contido (in, not in)	<code>q in L</code>	$O(n)$
Posição do valor	<code>i = q.index(v)</code>	$O(n)$
Inverte ordem	<code>q.reverse()</code>	$O(n)$
Mínimo	<code>v = min(q)</code>	$O(n)$
Máximo	<code>v = max(q)</code>	$O(n)$
Compara (==, !=)	<code>q1 == q2</code>	$O(n)$
Concatena	<code>q = q1 + q2</code>	$O(n)$
Tamanho	<code>len(q)</code>	$O(1)$
Iteração	<code>for v in q</code>	$O(n)$
Cópia	<code>q1 = q2.copy()</code>	$O(n)$
Repete	<code>k*q</code>	$O(kn)$

6.3 Dicionário (com tabela hash)

Um dicionário permite **localizar valores através de chaves**, ou seja, para cada valor v armazenado associamos uma chave exclusiva k . Desta forma, é possível localizar o valor v utilizando a chave k . Por exemplo, em um dicionário da língua portuguesa, a chave é a palavra k que desejamos localizar, e o valor v é o texto que descreve o significado da palavra.

Ao contrário das listas, que só permitem localizar um valor indicando a posição dele na lista (ou seja, a chave é um inteiro entre 0 e o tamanho da lista menos 1), os dicionários **permitem utilizar qualquer objeto imutável como chave**. As operações permitidas são apresentadas na tabela abaixo, e as complexidades fornecidas são de caso médio, e elas decorrem do fato do dicionário ser implementado com tabela hash.

Operação	Exemplo	Complexidade
Cria vazio	<code>d = {}</code>	$O(1)$
Cria inicializado	<code>d = {k1:v1, ..., kn:vn}</code>	$O(n)$
Inserir	<code>d[k] = v</code>	$O(1)$
Acessa	<code>v = d[k]</code>	$O(1)$
Altera	<code>d[k] = v</code>	$O(1)$
Remove	<code>v = d.pop(k)</code>	$O(1)$
Contido (in, not in)	<code>k in d</code>	$O(1)$
Menor chave	<code>k = min(d)</code>	$O(n)$
Maior chave	<code>k = max(d)</code>	$O(n)$
Compara (==, !=)	<code>d1 == d2</code>	$O(n)$
Tamanho	<code>len(d)</code>	$O(1)$
Iteração	<code>for k in d</code>	$O(n)$
Cópia	<code>d1 = d.copy()</code>	$O(n)$
Lista de chaves (cópia)	<code>l = list(d.keys())</code>	$O(n)$
Lista de valores (cópia)	<code>l = list(d.values())</code>	$O(n)$
Lista de tuplas (cópia)	<code>l = list(d.items())</code>	$O(n)$

- “Acessa”: lança uma exceção se a chave k não está no dicionário. No pseudocódigo basta testar se o valor retornado é “Não encontrado”.
- Para obter um valor default u quando a chave k não existe, utilize `v = d.get(k,u)`. Para armazenar o valor v na chave k apenas se a chave k ainda não existir no dicionário d , utilize `d.setdefault(k,v)`.
- “Contido”: verifica se o dicionário possui a chave k .
- “Iteração”: realiza uma iteração do for para cada chave k .
- “Compara”: retorna verdadeiro se e somente se os dois dicionários possuem as mesmas chaves, os mesmos valores e mesma associação entre chaves e valores.
- “Lista de tuplas”: retorna lista de tuplas com os elementos do dicionário, onde na 1a posição da tupla temos a chave e 2a posição o valor.

6.4 Conjunto (com tabela hash)

O tipo conjunto é similar a um conjunto na matemática: **elementos não podem ser repetidos, e não existe uma ordem entre os elementos**. Nas operações de dicionário, observamos que a operação “Contido” é realizada em tempo constante pela tabela hash. Desta forma, os conjuntos em python são implementados como tabelas hash, onde os valores são ignorados (usamos apenas as chaves). **Nos pseudocódigos, as operações com conjuntos também podem ser representadas em notação matemática.**

Operação	Exemplo	Complexidade
Cria vazio	<code>A = set()</code>	$O(1)$
Cria inicializado	<code>A = {v1, ..., vn}</code>	$O(n)$
Inserir	<code>A.add(v)</code>	$O(1)$
Remove	<code>A.remove(v)</code>	$O(1)$
Contido (in, not in)	<code>v in A</code>	$O(1)$
União	<code>C = A B</code>	$O(A + B)$
Interseção	<code>C = A & B</code>	$O(\min(A , B))$
Diferença	<code>C = A - B</code>	$O(A)$
Compara (==, !=)	<code>A == B</code>	$O(A)$
Subconjunto	<code>A <= B</code>	$O(A)$
Tamanho	<code>len(A)</code>	$O(1)$
Iteração	<code>for v in A</code>	$O(n)$
Cópia	<code>C = A.copy()</code>	$O(n)$
Cópia para lista	<code>l = list(A)</code>	$O(n)$

- “Compara”: executa em $O(1)$ se os tamanhos de A e B são distintos.
- “Subconjunto”: retorna verdadeiro se e somente se A está contido ou é igual a B .

6.5 Multiconjunto (com tabela hash)

Da mesma forma que em um conjunto, em um multiconjunto **não temos uma ordem entre os elementos**. Porém, ao contrário de um conjunto, em um multiconjunto **podemos ter elementos repetidos**. Uma forma eficiente de implementar o tipo multiconjunto é utilizar uma tabela hash onde os elementos armazenados são as chaves, e o valor associado a cada chave k é o número de ocorrências do elemento k no multiconjunto. Esta estrutura está implementada na classe Multiset do módulo python multiset (<https://pypi.org/project/multiset>). A sintaxe e a complexidade das operações na tabela estão de acordo com esta implementação.

Importação do módulo: `from multiset import Multiset`

Operação	Exemplo	Complexidade
Cria vazio	<code>A = Multiset()</code>	$O(1)$
Cria inicializado	<code>A = Multiset({v1:c1, ..., vn:cn})</code>	$O(n)$
Define # cópias	<code>A[v] = c</code>	$O(1)$
Acessa # cópias	<code>c = A[v]</code>	$O(1)$
Contido (in, not in)	<code>v in A</code>	$O(1)$
União	<code>C = A B</code>	$O(A + B)$
Interseção	<code>C = A & B</code>	$O(\min(A , B))$
Diferença	<code>C = A - B</code>	$O(A)$
Compara (==, !=)	<code>A == B</code>	$O(A)$
Subconjunto	<code>A <= B</code>	$O(A)$
Tamanho	<code>len(A)</code>	$O(1)$
Iteração	<code>for v in A</code>	$O(n)$
Cópia	<code>C = A.copy()</code>	$O(n)$
Cópia para lista	<code>l = list(A)</code>	$O(n)$

- “Acessa # cópias”: retorna o número de cópias de v se v está no multiconjunto, ou zero se o elemento v não está no conjunto.
- “Subconjunto”: retorna verdadeiro se e somente se o número de cópias de cada elemento v em A é menor ou igual ao número de cópias de v em B .
- “Tamanho”: retorna o total de elementos considerando as repetições.

6.6 Conjuntos disjuntos (com union-find)

No tipo conjunto é possível fazer a união em tempo $O(\min(|A|, |B|))$ copiando os elementos do menor conjunto para o maior conjunto, com a desvantagem de perder o conjunto original. Ainda assim, o tempo de execução da união depende do número de elementos nos conjuntos.

Existe uma situação em que a **união pode ser feita em tempo “praticamente” constante: quando os conjuntos são mantidos disjuntos, ou seja, sem interseção**. Neste caso dispomos da estrutura de dados union-find. Uma implementação em python está disponível em <https://github.com/deehzee/unionfind/blob/master/unionfind.py> A tabela abaixo fornece a notação e a complexidade de cada operação nesta implementação.

Importação do módulo: `from unionfind import UnionFind`

Operação	Exemplo	Complexidade
Cria vazio	<code>U = UnionFind()</code>	$O(1)$
Cria inicializado	<code>U = UnionFind([e1,...,en])</code>	$O(n)$
Inserir	<code>U.add(e)</code>	$O(1)$
União	<code>U.union(e1,e2)</code>	$O(1)$
Find	<code>U.find(e)</code>	$O(1)$
Conectados	<code>U.connected(e1,e2)</code>	$O(1)$
Componente	<code>U.component(e)</code>	$O(n)$
Componentes	<code>U.components()</code>	$O(mn)$
Contido (in, not in)	<code>e in U</code>	$O(1)$
Tamanho	<code>len(U)</code>	$O(1)$
Iteração	<code>for e in U</code>	$O(n)$
Cópia	<code>U1 = UnionFind(list(U))</code>	$O(n)$
Cópia para lista	<code>l = list(U)</code>	$O(n)$

- “Cria inicializado” cria um conjunto para cada elemento.
- “Tamanho”: número de elementos (não de conjuntos).
- “Inserir”: acrescenta um novo elemento e , e cria um novo conjunto contendo apenas este novo elemento.
- “Find”: retorna o identificador do conjunto que contém o elemento e . Este identificador é utilizado para verificar se dois elementos estão no mesmo conjunto.
- “Conectados”: retorna verdadeiro apenas se os elementos e_1 e e_2 estão no mesmo conjunto.
- “União”: realiza a união do conjunto que contém e_1 com o conjunto que contém e_2 .
- “Componente”: retorna o conjunto que contém o elemento e .
- “Componentes”: retorna uma lista com todos os conjuntos. A complexidade é $O(mn)$, onde m é o número de conjuntos e n é o total de elementos.

6.7 Conjunto ordenado (com árvore balanceada)

Suponha que seu algoritmo precisa realizar **várias operações de inserção/remoção e também determinar várias vezes qual é o i -ésimo menor elemento**. Para comparar a complexidade das implementações abaixo, considere o cenário onde vamos inserir n elementos, e em seguida fazer n operações de inserção/remoção e depois n operação de busca pelo i -ésimo menor elemento.

Array não ordenado Gastamos $O(1)$ por inserção, $O(n)$ por remoção, e $O(n)$ por busca do i -ésimo menor, totalizando $O(n^2)$.

Array ordenado Gastamos $O(n \log n)$ para inserir e ordenar os n primeiros elementos, $O(n)$ por inserção/remoção, e $O(1)$ por busca do i -ésimo menor, totalizando $O(n^2)$.

Árvore balanceada Gastamos $O(\log n)$ em todas as operações, totalizando $O(n \log n)$.

Concluimos que neste caso a implementação com **árvore balanceada produz a menor complexidade total**. A sintaxe das operações apresentadas na tabela corresponde à classe `SortedSet` do módulo `python sortedcontainers` (<https://pypi.org/project/sortedcontainers/>). As complexidades listadas são de implementações de árvore balanceada, embora a classe `SortedSet` esteja implementada com árvore-B, resultando em complexidade amortizadas $O(\sqrt[3]{n})$ para inserção/remoção/busca. Apesar disso, como as constantes da complexidade da árvore-B são menores que de árvores balanceadas, os desenvolvedores afirmam que na prática o desempenho da árvore-B é melhor (<http://www.grantjenks.com/docs/sortedcontainers/performance-scale.html>).

Importação do módulo: `from sortedcontainers import SortedSet`

Operação	Exemplo	Complexidade
Cria vazio	<code>S = SortedSet()</code>	$O(1)$
Cria inicializado	<code>S = SortedSet([v1,...,vn])</code>	$O(n \log n)$
Inserir	<code>S.add(v)</code>	$O(\log n)$
Remove	<code>S.remove(v)</code>	$O(\log n)$
Contido (in, not in)	<code>v in S</code>	$O(1)$
i -ésimo menor	<code>S[i]</code>	$O(\log n)$
Mínimo	<code>v = S[0]</code>	$O(\log n)$
Máximo	<code>v = S[-1]</code>	$O(\log n)$
União	<code>C = A B</code>	$O(A + B)$
Interseção	<code>C = A & B</code>	$O(\min(A , B))$
Diferença	<code>C = A - B</code>	$O(A)$
Compara (==, !=)	<code>A == B</code>	$O(A)$
Subconjunto	<code>A <= B</code>	$O(A)$
Tamanho	<code>len(S)</code>	$O(1)$
Iteração	<code>for v in S</code>	$O(n)$
Cópia	<code>C = S.copy()</code>	$O(n)$
Cópia para lista	<code>l = list(S)</code>	$O(n)$

- “ i -ésimo menor”: retorna o i -ésimo menor elemento começando da posição 0, ou seja, o menor elemento está na posição 0, o segundo menor na posição 1, assim sucessivamente. Portanto, o maior elemento está na última posição, que pode ser acessada com `S[-1]`.
- A operação “Contido” pode ser realizada em $O(1)$ mantendo uma tabela hash auxiliar, por exemplo.

6.8 Fila de prioridade (com heap)

Na fila de prioridade estamos interessados **apenas na operação de inserção de elemento, e na operação de remoção do menor elemento**. Por ser **mais restrita** que o conjunto ordenado, a fila de prioridade consegue realizar estas operações de forma **mais eficiente**. Note que o conjunto ordenado permite adicionalmente remover qualquer elemento e localizar o i -ésimo menor. A tabela mostra as complexidades da implementação das operações de fila de prioridade utilizando heap. Adotamos como notação as funções disponíveis no módulo `python heapq` (<https://docs.python.org/3/library/heapq.html>).
Importação do módulo: `from heapq import *`

Operação	Exemplo	Complexidade
Cria vazia	<code>h = []</code>	$O(1)$
Cria inicializada	<code>h = [v1,...,vn]</code> <code>heapify(h)</code>	$O(n)$ $O(n)$
Inserir v na heap h	<code>heappush(h,v)</code>	$O(\log n)$
Remove menor	<code>v = heappop(h)</code>	$O(\log n)$
Mínimo	<code>v = h[0]</code>	$O(1)$
Aumenta chave	<code>increase_key(h,v,k)</code>	$O(\log n)$
Reduz chave	<code>decrease_key(h,v,k)</code>	$O(\log n)$

- A heap é mantida em uma lista, sem dados adicionais. Portanto, para criar uma heap vazia simplesmente declaramos uma lista vazia, e para criar uma heap inicializada chamamos `heapfy` para transformar a lista em uma heap (esta operação apenas reposiciona elementos da lista).
- Todas as operações de lista também são permitidas na heap, pois a heap é uma lista. Porém, modificações na lista podem fazê-la deixar de ser uma heap.
- Se os elementos são números, para ter uma heap que acesse e remova o maior elemento (ao invés do menor elemento) inverta o sinal dos números armazenados. Esta estratégia também é válida quando os elementos são tuplas, e o primeiro elemento é um número que determina a ordem das tuplas (veja a seção “Comparação de tuplas”). Quando usamos tuplas, a primeira posição é chamada de **chave**.
- A operação `increase_key` aumenta para k o valor da chave de v na heap h . Da mesma forma, a operação `decrease_key` reduz para k o valor da chave de v na heap h . As duas operações exigem tempo $O(\log n)$, pois a mudança na chave k provoca um reposicionamento da tupla (k,v) na heap ao longo da altura da árvore. Estas operações estão implementadas no módulo `heapq` nos métodos “privados” `_siftup(h,p)` (correspondente ao `increase_key`) e `_siftdown(h,s,p)` (correspondente ao `decrease_key`), onde p é a posição na heap h do elemento que teve a chave modificada, e s é a posição inicial (que podemos fixar em zero) (veja o código-fonte: <https://hg.python.org/cpython/file/2.7/Lib/heapq.py>). Podemos usar um dicionário auxiliar para obter em $O(1)$ a posição de uma dada tupla na heap h . A desvantagem de usar métodos “privados” (métodos iniciando com underscore) é que versões futuras da biblioteca podem remover estes métodos.
- Vantagens da heap em relação ao conjunto ordenado:
 - Menor complexidade na criação inicializada e na consulta ao menor (sem remoção dele).
 - Embora tenha a mesma complexidade na inserção e remoção, a heap é mais rápida na prática, pois tem implementação mais simples e constantes menores na complexidade. Veja esta comparação de desempenho: <https://medium.com/@ssbothwell/comparing-sorted-containers-in-python-a2c41624bc84>

Comparação de tuplas

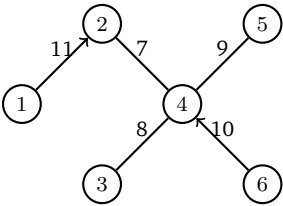
Podemos armazenar tuplas nas estruturas, obtendo assim mais flexibilidade, pois as tuplas permitem armazenar várias informações em cada elemento. Por exemplo, podemos colocar em uma heap tuplas contendo o nome, a matrícula e a nota de cada aluno.

Se tuplas são armazenadas nas estruturas, precisamos saber como elas são comparadas. Isto é particularmente importante nas estruturas “conjunto ordenado” e “fila de prioridade”. Tuplas são comparadas **da menor posição para a maior posição**. Ex.: `(4,'def') < (5,'abc')` é verdadeiro, pois o elemento da 1ª posição é menor na tupla da esquerda. Esta forma de comparar tuplas é chamada de **ordem lexicográfica**.

6.9 Grafo (com lista de adjacências)

Uma listas de adjacência pode ser representada como um dicionário de conjuntos. Desta forma, $G[i]$ contém o conjunto de vértices adjacentes a i . Se o grafo possui pesos nas arestas, basta representar como um dicionário de dicionários. Neste caso, $G[i][j]$ armazena o peso da aresta.

Considere o exemplo abaixo. Arestas sem direção indicam as que as duas direções são permitidas.



```
G = {}
G[1] = {2:11}
G[2] = {4:7}
G[3] = {4:8}
G[4] = {2:7, 3:8, 5:9}
G[5] = {4:9}
G[6] = {4:10}
```

Operação	Exemplo	Complexidade
Cria vazio	$G = \{\}$	$O(1)$
Insera vértice	$G[i] = \{\}$	$O(1)$
Insera aresta	$G[i][j] = v$	$O(1)$
Remove vértice	$G.pop(i)$	$O(1)$
Remove aresta	$G[i].pop(j)$	$O(1)$
Iteração nos vértices	$\text{for } i \text{ in } G$	$O(n)$
Iteração nos adjacentes	$\text{for } j \text{ in } G[i]$	$O(G[i])$

6.10 Exemplos

Os exemplos a seguir estão explicados no vídeo “tipos abstratos de dados (exemplos)”, mas vou fornecer os pseudocódigos utilizando a notação e complexidades que foram apresentadas nestas notas de aula.

Algoritmo: Parsing($s[1..n]$)

Entrada: String $s[1..n]$ cujos caracteres são indexados de 1 até n .
A string s contém apenas os símbolos (,), [,], { e }.

Saída : Retorna lista com inteiros que associam cada abertura com o fechamento correspondente em s , ou falso se s é inválida.

```
1 (P, prox_id, ids) ← ([], 0, []) // O(1)
2 for i de 1 até n do // O(n)
3   if s[i] = '(' or s[i] = '[' or s[i] = '{' then // O(n)
4     ids.append(prox_id) // O(n) · O(1) = O(n)
5     P.append((s[i], prox_id)) // O(n) · O(1) = O(n)
6     prox_id ← prox_id + 1 // O(n)
7   else // O(n)
8     if len(P) = 0 then return false // O(n)
9     (topo_char, topo_id) ← P.pop() // O(n) · O(1) = O(n)
10    if s[i] e topo_char não são do mesmo tipo then // O(n)
11      return false // O(1)
12    ids.append(topo_id) // O(n) · O(1) = O(n)
13 return ids // O(1)
```

Algoritmo: Frequência_palavras($s[1..n]$)

Entrada: Lista $s[1..n]$ contendo n palavras, indexadas de 1 até n .

Saída : Dicionário D, onde $D[w]$ é o número de vezes que a palavra w aparece em s .

```
1 D ← {} // O(1)
2 for w in s do // O(n)
3   D[w] ← D.get(w,0) + 1 // O(n) · O(1) = O(n)
4 return D // O(1)
```

Leitura complementar

Capítulos 3 e 4 do Skiena, e capítulos 6, 10, 11, 12, 13 e 21 do Cormen.

Livro online “Estruturas de Dados Abertos”:
<https://github.com/jaraujouerj/Estruturas-de-Dados-Abertos>

Algoritmo: Prim(G,w)

Entrada: Grafo não direcionado G . $G[u][v]$ é o peso da aresta $\{u,v\}$.

Saída : Dicionário π , onde $\pi[u]$ é o nó predecessor do nó u em uma árvore geradora mínima de G .

// inicialização das estruturas

```
1 s ← nó qualquer de G // O(1)
2 ( $\pi$ , key, Q) ← ({}, {}, []) // O(1)
3 for v in G do // O(n)
4    $\pi[v] \leftarrow \text{null}$  // O(n)
5   if  $v \neq s$  then // O(n)
6     key[v] ←  $\infty$  // O(n)
7     Q.append((key[v], v)) // O(n)
```

8 key[s] ← 0 // O(1)

9 Q.append((key[s], s)) // O(1)

10 heapify(Q) // O(n)

// seleção das arestas da árvore geradora mínima

```
11 while len(Q) > 0 do // O(n)
12   (k,u) ← heappop(Q) // O(n) · O(log n) = O(n log n)
13   key.pop(u) // O(n) · O(1) = O(n)
14   for v in G[u] do // O( $\sum_v d_v$ ) = O(m)
15     if v in key and  $G[u][v] < \text{key}[v]$  then // O(m)
16       decrease_key(Q, v,  $G[u][v]$ ) // O(m)O(log n) = O(m log n)
17        $\pi[v] \leftarrow u$  // O(m)
```

18 return π // O(1)

// Complexidade total: $O((n + m) \log n)$

Algoritmo: Kruskal(G,w)

Entrada: Grafo não direcionado G . $G[u][v]$ é o peso da aresta $\{u,v\}$.

Saída : Lista E de arestas de uma árvore geradora mínima de G .

// inicialização das estruturas

```
1 U ← UnionFind([v in G ]) // O(n)
2 E ← set() // O(1)
3 Q ← [] // O(1)
4 for u in G do
5   for v in G[u] do // O(m)
6     Q.append((G[u][v], (u, v))) // O(m)
```

7 heapify(Q) // O(m)

// seleção das arestas da árvore geradora mínima

```
8 while len(Q) > 0 do // O(m)
9   (k,(u,v)) ← heappop(Q) // O(m) · O(log m) = O(m log m)
10  if U.find(u) = U.find(v) then // O(m)
11    U.union(u,v) // no max n – 1 uniões: O(n) · O(1) = O(n)
12    E.append((u,v)) // O(n) · O(1) = O(n)
```

13 return E // O(1)

// Complexidade total: $O(n + m \log m)$

(tradução de <http://opendatastructures.org>)

Complexidades das operações em python:

- <https://www.ics.uci.edu/~pattis/ICS-33/lectures/complexitypython.txt>
- <https://wiki.python.org/moin/TimeComplexity>